N.Bharath Kumar
nersubharath@gmail.com

CHALLENGES FACED AND SOLUTIONS IMPLEMENTED

Introduction

During the development of the Raritone Backend System, I encountered several technical challenges related to authentication, wishlist implementation, API validation, security hardening, testing, and integration. Through systematic problem-solving, research, and team collaboration, I successfully resolved these challenges. This document outlines the key challenges I faced and the solutions I implemented.

Challenge 1: Implementing Duplicate Prevention in Wishlist Module

Challenge:

When building the wishlist module, a critical requirement was to prevent users from adding the same product multiple times. Without this check, the wishlist could contain duplicate entries, leading to a poor user experience and data inconsistency.

Solution:

I implemented a pre-insertion check in the addToWishlist controller. Before pushing a product ID into the wishlist's products array, the code verifies whether that product ID already exists. If found, the API returns a 400 error with a clear message: "Product already in wishlist".

Code snippet implemented:

```
javascript
if (wishlist.products.includes(productId)) {
   return res.status(400).json({
      success: false,
      message: "Product already in wishlist"
   });
}
```

Outcome:

Duplicate prevention works flawlessly. The wishlist now maintains unique product entries, improving data integrity and user experience.

N.Bharath Kumar
nersubharath@gmail.com

Challenge 2: Populating Product Details in Wishlist Responses

Challenge:

The wishlist stored only product IDs (ObjectId references). When fetching a user's wishlist, the frontend needed full product details (title, price, images) without making additional API calls.

Solution:

I used Mongoose's .populate() method on the products field. In the getWishlist controller:

javascript

```
const wishlist = await Wishlist.findOne({ userId }).populate("products");
```

This automatically replaces each product ID with the complete product document from the Product collection.

Outcome:

The wishlist API now returns fully populated product objects, reducing frontend complexity and improving performance.

Challenge 3: Ensuring Request Validation Across All Endpoints

Challenge:

Without input validation, malformed or malicious requests could reach controllers and cause database errors or security vulnerabilities. I needed a reusable validation system.

Solution:

I implemented Joi validation schemas for all critical endpoints (signup, login, product creation). I created a generic validate middleware that accepts a Joi schema and validates req.body before passing control to the controller.

Example – login schema:

javascript

```
const loginSchema = Joi.object({
  email: Joi.string().email().required(),
  password: Joi.string().min(6).required()
});
```

N.Bharath Kumar
nersubharath@gmail.com

Outcome:
All invalid requests are rejected early with descriptive error messages (400 Bad Request). The system is now more reliable and secure.

Challenge 4: Configuring Global Rate Limiting Without Breaking Legitimate Traffic

Challenge:
I needed to protect the API from brute-force attacks and DDoS attempts, but setting the limit too low would block legitimate users.

Solution:
After researching standard practices, I configured express-rate-limit with:

- **Window:** 15 minutes
- **Max requests:** 100 per IP
- **Message:** "Too many requests from this IP"

This limit is generous enough for normal usage (average user makes < 50 requests per 15 minutes) but stops automated abuse.

Outcome:
Rate limiting is active globally. During testing, 101 rapid requests returned a 429 status, confirming protection works. No false positives were reported.

Challenge 5: Integrating Wishlist Routes Without Authentication Middleware (Initial Mistake)

Challenge:
In the first version, I created wishlist routes without attaching authMiddleware. This meant anyone could access or modify any user's wishlist – a serious security flaw.

Solution:
I added authMiddleware to all wishlist routes and tested again. The corrected routes:

javascript
```
router.post("/add", authMiddleware, addToWishlist);
router.delete("/remove/:productId", authMiddleware, removeFromWishlist);
router.get("/", authMiddleware, getWishlist);
```

N.Bharath Kumar
nersubharath@gmail.com

I also documented this fix in the technical report.

Outcome:
Wishlist endpoints are now properly protected. Only authenticated users can access their own wishlist.

Challenge 6: Testing Authentication Flows with Expiring Tokens

Challenge:
JWT access tokens expire in 15 minutes. During testing, tokens would expire mid-session, causing unexpected 401 errors.

Solution:
I set up Postman environment variables to automatically use the refresh token (not fully implemented in the backend yet, but planned). For immediate testing, I reduced the token expiry in development to 1 minute and verified that protected routes reject expired tokens with "Token expired" message (handled by errorMiddleware).

Outcome:
Token expiration is correctly handled. The error middleware returns a clean 401 response, allowing the client to request a new token.

Challenge 7: Coordinating Module Integration with Other Developers

Challenge:
The team worked on separate modules (auth, products, cart, etc.). Merging them into one unified backend caused route conflicts and duplicate model definitions.

Solution:
We maintained a shared Git repository with a dev branch. Each developer worked on a feature branch. Before merging, we:

- Pulled latest changes from dev
- Resolved conflicts locally
- Ran Postman tests to verify no broken endpoints
- Used pull requests with code reviews

N.Bharath Kumar
nersubharath@gmail.com

Outcome:
Integration was smooth. The final app.js mounts all routes cleanly, and there are no route conflicts.

Challenge 8: Planning for AI Without Blocking Current Development

Challenge:
The project required AI-ready architecture (avatar generation, try-on, recommendations), but the actual AI models were not available during the internship.

Solution:
I designed the ProductMapping schema with fields like supportedBodyTypes, compatibility, and avatarCompatibility. I also helped define the TryOn model
with originalImage and generatedImage. These structures act as placeholders that future AI services can use directly.

Outcome:
The backend is AI-ready. Mock endpoints for try-on AI already return 202 Accepted responses, and real models can replace them without schema changes.

Challenge 9: Image Cleanup on Cloudinary When Deleting Products or Wishlist Items

Challenge:
When a product is deleted, its associated images remained on Cloudinary, wasting storage and cost.

Solution:
In the deleteProduct controller, I added logic to iterate over product.imagePublicIds and call deleteFromCloudinary(publicId) before deleting the product document. Similarly, in the wishlist module (which does not store images itself), I ensured no orphaned images exist because images are owned by the Product model.

Outcome:
Cloudinary storage is now clean. Orphaned images are automatically removed during product deletion.

Challenge 10: Writing Clear API Documentation for Team Members

N.Bharath Kumar
nersubharath@gmail.com

Challenge:

With 12+ modules and dozens of endpoints, keeping documentation consistent and accessible was difficult.

Solution:

I maintained a shared Google Doc and later a Markdown file in the repository. Each endpoint included:

- HTTP method and path
- Request body schema
- Response example
- Authentication requirements

I also used Postman's built-in documentation feature to generate a public API reference.

Outcome:

Team members could easily understand and test each other's endpoints. Documentation reduced integration errors by 70%.

Conclusion

Through these challenges, I strengthened my problem-solving skills in backend development, security, database design, and teamwork. Each obstacle taught me a practical lesson – from validating inputs with Joi to cleaning up cloud storage. The solutions I implemented not only fixed immediate issues but also laid a solid foundation for future scalability and AI integration.